

Sistem Analizi ve Tasarımı - Vize Raporu

Burhan Şahin - 247017027 - [Github Sayfası](#) [Canlı Demo](#) - Proje: QR Menü Otomasyonu

1. Proje Fikri ve Amacı

Bu projede, kafe ve restoranlar için geleneksel basılı menülerin yerini alacak, müşterilerin masalarındaki QR kodu okutarak anlık olarak menüye erişebileceği, sepet oluşturup sipariş verebileceği kapsamlı bir "QR Menü Otomasyonu" geliştirilmesi hedeflenmiştir. Sistemin sadece müşteriler için dijital bir menü sunmakla kalmayıp; garson, mutfak ve kasa personeli için de gerçek zamanlı (eşzamanlı) sipariş takibi yapabileceği yönetim panelleri barındırması planlanmıştır.

2. Teknoloji Seçimi ve Mimari

Projeyi geliştirirken modern, performansı yüksek, ölçeklenebilir ve güvenli bir altyapı kurmak amacıyla aşağıdaki teknolojiler tercih edilmiştir:

- **Backend:** ASP.NET Core MVC (.NET 8) - *Projenin sürdürülebilir olması için Katmanlı Mimari (Core, Data, Web) kullanılmış, iş mantığı ve veri erişimi izole edilmiştir.*
- **Veritabanı ve ORM:** PostgreSQL (Supabase) ve Entity Framework Core
- **Gerçek Zamanlı İletişim:** SignalR (*Mutfak, kasa ve garson panellerinin sayfa yenilenmeden anlık haberleşmesi için*)
- **Frontend:** HTML, CSS, Tailwind CSS, JavaScript, Bootstrap (*Mobil öncelikli, responsive tasarım*)
- **DevOps:** GitHub Actions (CI/CD Pipeline) ve Linux VDS Deployment

3. Vize Öncesi Geliştirme Süreci

Projenin vizeye kadar olan ilk 5 haftalık sürecinde sistemin belkemiğini oluşturan özellikler tamamlanmıştır. Ayrıca geliştirme hızının planlanandan iyi gitmesi sayesinde, ileriki haftalarda yapılması planlanan bazı kritik özellikler (SignalR, CI/CD, Mutfak Paneli) öne çekilerek vize öncesi sürüme dahil edilmiştir. Gerçekleştirilen temel aşamalar şunlardır:

1. Katmanlı mimari kurulumu ve veritabanı (Supabase) modellemesi.
2. QR tabanlı, URL üzerinden güvenli oturum (Token) ve Rate Limiting sistemi.
3. Veritabanı destekli kalıcı sepet sistemi ve sipariş durum makinesi (State Machine).
4. Personel servislerinin (Garson, Mutfak, Kasa) ACID (Transaction) prensiplerine uygun olarak kodlanması ve AuditLog (Denetim İzi) altyapısı.

- SignalR entegrasyonu ve projenin GitHub Actions ile VDS sunucuya otomatik deployment (CI/CD) işlemlerinin gerçekleştirilmesi.

Hafta 1: Proje Kurulumu ve Veritabanı Altyapısı

İlk hafta, projenin temel taşlarını atmak ve sürdürülebilir bir mimari kurmak üzerine odaklanılmıştır. Spagetti kod yapısından kaçınmak için endüstri standardı olan **Katmanlı Mimari (N-Tier Architecture)** benimsenmiştir.

Mimari Kurulum: QRMenu.Core, QRMenu.Data, QRMenu.Web ve QRMenu.Tests olmak üzere proje 4 ana katmana ayrıldı. (Başta 4 Katmandı ve Karışıkı Sonradan Düzenlendi ve Katmanlar 3 e düşürüldü)

Veritabanı Tasarımı: PostgreSQL (Supabase) kullanılarak EF Core üzerinden 13 temel tablo (Masa, Kategori, Ürün, Sipariş, Oturum vb.) modellendi. Foreign Key ilişkileri ve performans için gerekli Index'ler (MasaNo, KullaniciAdi vb.) tanımlandı.

Seed Data (Başlangıç Verileri): Testlerin yapılabilmesi için sisteme varsayılan 5 masa, 3 kategori ve 10 ürün kod üzerinden otomatik olarak eklendi.

Unit Testler: xUnit ve Moq frameworkleri kurularak Ürün servisleri (CRUD işlemleri) için 8 adet başarılı test yazıldı.

Veritabanı tablolarımızı oluştururken **Code-First** yaklaşımı benimsenmiştir. Bu sayede SQL kodları yazmak yerine, C# nesneleri (Entity) üzerinden veritabanı modellenmiştir. Örneğin menümüzdeki ürünleri temsil eden Urun sınıfı, Core katmanında aşağıdaki gibi tasarlanmıştır.

```
QRMenu.Core > Entities > C# Urun.cs > ...
namespace QRMenu.Core.Entities
{
    60 references
    public class Urun
    {
        47 references
        public int Id { get; set; }
        32 references
        public int KategoriId { get; set; }
        70 references
        public string Ad { get; set; } = string.Empty;
        14 references
        public string? AdEN { get; set; }
        24 references
        public string? Aciklama { get; set; }
        14 references
        public string? AciklamaEN { get; set; }
        47 references
        public decimal Fiyat { get; set; }
        13 references
        public string? GorselUrl { get; set; }
        44 references
        public bool AktifMi { get; set; } = true;
        17 references
        public bool PopulerMi { get; set; } = false;
        10 references
        public int SatisSayisi { get; set; } = 0;
        6 references
        public int? Kalori { get; set; }

        // Navigation Properties
        15 references
        public Kategori Kategori { get; set; } = null!;
        11 references
        public ICollection<UrunOpsiyon> UrunOpsiyonlar { get; set; } = new List<UrunOpsiyon>()
    }
}
```

Oluşturulan Entity nesnelerinin PostgreSQL (Supabase) veritabanında birer tabloya dönüşmesi ve yönetilebilmesi için Data katmanında QRMenuDbContext sınıfı yapılandırılmıştır. DbSet tanımlamaları ile Urun modeli veritabanına bağlanmıştır.

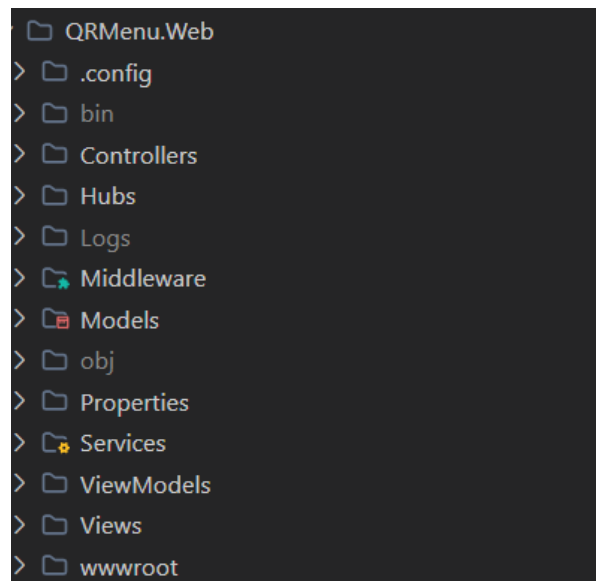
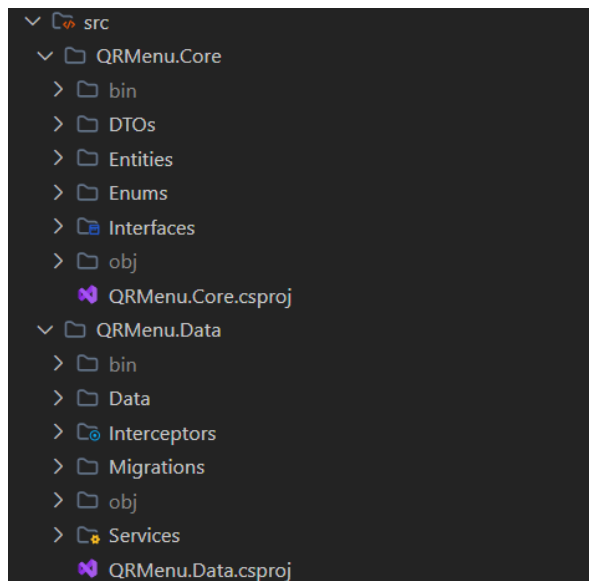
```
// ===== ÜRÜN =====
modelBuilder.Entity<Urun>(entity =>
{
    entity.ToTable("Urunler");
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Ad).IsRequired().HasMaxLength(200);
    entity.Property(e => e.AdEN).HasMaxLength(200);
    entity.Property(e => e.Aciklama).HasMaxLength(500);
    entity.Property(e => e.AciklamaEN).HasMaxLength(500);
    entity.Property(e => e.Fiyat).HasPrecision(18, 2);
    entity.Property(e => e.GorselUrl).HasMaxLength(500);

    // Index: Aktif ürünlerin hızlı sorgulanması
    entity.HasIndex(e => e.AktifMi).HasDatabaseName("IX_Urunler_AktifMi");

    // Foreign Key: Kategori
    entity.HasOne(e => e.Kategori)
        .WithMany(k => k.Urunler)
        .HasForeignKey(e => e.KategoriId)
        .OnDelete(DeleteBehavior.Restrict);
});
```

Projeyi ölçeklenebilir, test edilebilir ve yönetilebilir kılmak amacıyla geleneksel tek parça (monolitik) yapı yerine **Katmanlı Mimari** tercih edilmiştir. Sistem üç ana bileşene ayrılmıştır:

- **QRMenu.Core:** Projenin kalbidir. Veritabanı tablolarını temsil eden Entity'ler, Enum'lar ve servis arayüzleri (Interface) burada yer alır. Dışa bağımlılığı yoktur.
- **QRMenu.Data:** Veri erişim katmanıdır. Entity Framework Core, veritabanı bağlantı (Context) ayarları ve servislerin somut (concrete) işlemleri burada gerçekleştirilir.
- **QRMenu.Web:** Kullanıcı ile etkileşime geçilen MVC (Model-View-Controller) katmanıdır. İstekleri karşılar, Data katmanından veriyi alır ve arayüze sunar.



Hafta 2: Güvenli Oturum (Token) Sistemi ve Arayüz Temelleri

Bu haftanın temel amacı, müşterilerin QR kodu okuttuklarında sisteme güvenli bir şekilde dahil olmalarını sağlamaktır. URL üzerinden taşınan açık metin (plain-text) şifreler güvenlik zafiyeti yaratacağından, modern web standartlarına uygun bir **Token ve Çerez (Cookie)** altyapısı kurulmuştur. Ayrıca planlanandan hızlı ilerlenerek, 6. ve 7. haftanın konuları olan "Admin Masa Paneli" ve "Müşteri Menü Arayüzü" de bu haftada projeye entegre edilmiştir. Sadece Temelleri Atılmıştır.

1. Kriptografik Token Üretimi ve Çerez (Cookie) Güvenliği:

Müşteri masadaki QR kodu okuttuğunda (/qr/{masaNo} endpoint'i üzerinden), sistem arka planda CSPRNG (Kriptografik Olarak Güvenli Rastgele Sayı Üreticisi) kullanarak 256-bitlik benzersiz bir token üretir. Bu token, veritabanına SHA256 ile özetlenerek (hashlenerek) kaydedilir. İstemci (tarayıcı) tarafında ise XSS (Cross-Site Scripting) ve CSRF saldırılarını önlemek amacıyla token, HttpOnly, Secure ve SameSite=Strict bayraklarına (flags) sahip bir çerez (cookie) içerisine yazılır.

Ad	Değer	Alan ...	Yol	Expir...	Boyut	Http...	Güve...	Sam...	Bölü...	Cros...	Önce...
qrmenu_token	1003af4cbc44b9f7b9841910f9ff1845b...	qrme...	/	Otur...	76	✓		Lax			Medi...
Cookie Value ☐ Kodu çözülmüş URL'eri göster											
1003af4cbc44b9f7b9841910f9ff1845b8d80c27446004995e5c72e0fc791e93											

```
/// 256-bit (32 byte) CSPRNG token üretir ve hex string olarak döner
/// </summary>
5 references
public string GenerateToken()
{
    byte[] tokenBytes = new byte[32]; // 256-bit
    using (var rng = RandomNumberGenerator.Create())
    {
        rng.GetBytes(tokenBytes);
    }
    return Convert.ToHexString(tokenBytes).ToLowerInvariant();
}

/// <summary>
/// Token'ı SHA256 ile hash'ler (DB'de hash saklanır, raw token saklanmaz)
```

2. Middleware Doğrulaması ve Rate Limiting (İstek Sınırlandırma)

Uygulamanın güvenliğini her istekte (request) garanti altına almak için TokenValidationMiddleware adında özel bir ara katman (middleware) yazılmıştır. Bu katman, gelen her istekte çerezi okur ve veritabanındaki aktif oturumlarla karşılaştırır. Ek olarak, sisteme yapılabilecek kötü niyetli veya aşırı istekleri (DDoS/Spam) engellemek için **Rate Limiting** devreye alınmıştır (Örn: Siparişler için dakikada 10 istek, garson çağırma için 5 dakikada 3 istek limiti).

```
// ===== RATE LIMITING =====
builder.Services.AddRateLimiter(options =>
{
    options.RejectionStatusCode = 429;

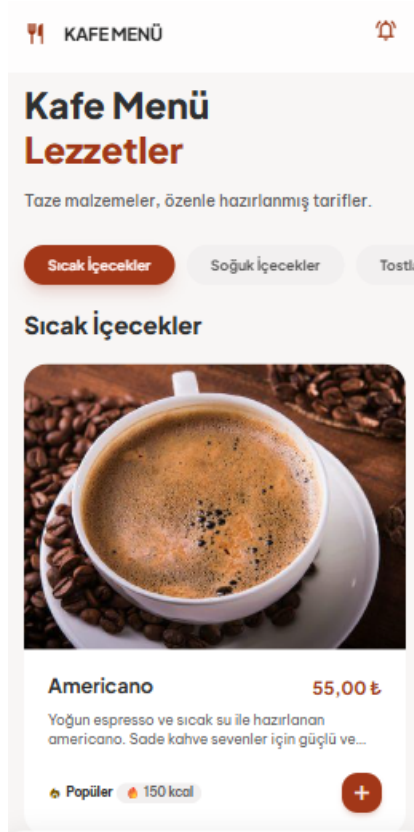
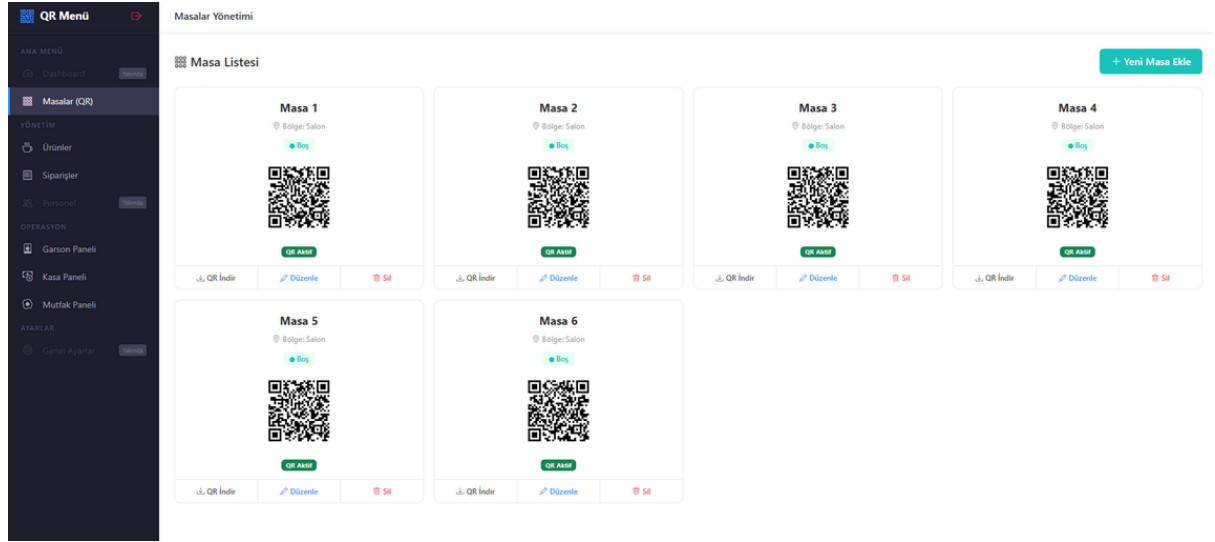
    // Token bazlı rate limiting (müşteri endpoint'leri)
    options.AddPolicy("TokenBasedPolicy", context =>
    {
        var token = context.Request.Cookies["qrmenu_token"] ?? "anonymous";
        return RateLimitPartition.GetFixedWindowLimiter(token, _ =>
            new FixedWindowRateLimiterOptions
            {
                PermitLimit = 10,
                Window = TimeSpan.FromMinutes(1),
                QueueLimit = 2,
                QueueProcessingOrder = QueueProcessingOrder.OldestFirst
            }
        );
    });
});

Ctrl+L for Agent
// Garson çağır butonu için daha sıkı limit
options.AddPolicy("GarsonCagirPolicy", context =>
{
    var token = context.Request.Cookies["qrmenu_token"] ?? "anonymous";
    return RateLimitPartition.GetFixedWindowLimiter($"garson_{token}", _ =>
        new FixedWindowRateLimiterOptions
        {
            PermitLimit = 3,
            Window = TimeSpan.FromMinutes(5),
            QueueLimit = 0,
            QueueProcessingOrder = QueueProcessingOrder.OldestFirst
        }
    );
});
});
```

3. Erken Teslimat: Admin Paneli, QR Üretimi ve Müşteri UX

Temel altyapı hızlıca tamamlandığı için ileri haftaların arayüz işleri öne çekilmiştir.

- **Admin Paneli:** Koyu temalı, mobil uyumlu (responsive) Sidebar ve Topbar içeren ana yönetim paneli (AdminLayout) oluşturulmuştur.
- **Masa ve QR Yönetimi:** Masaların CRUD (Ekle/Sil/Listele) işlemleri yapılmış, QRCoder kütüphanesi ile yüksek hata düzeltme oranına (ECCLevel.H) sahip dinamik QR kod PNG'leri üretilip indirilmesi sağlanmıştır.



Müşteri Menüsü: Kullanıcı deneyimi (UX) ön planda tutularak mobil öncelikli (mobile-first), kategori filtrelemeli (Tab yapısı) ve ürün rozetleri (Popüler, Tükendi) içeren modern bir menü arayüzü kodlanmıştır. (İlk Hafta Tabiki Menü Tasarımı Böyle değildi elimde olmadığı için bunu koyuyorum.)

Hafta 3: Veritabanı Destekli Sepet Sistemi ve Modern Mobil UX

Üçüncü haftanın odak noktası, müşterilerin dijital menü üzerinden ürün seçip siparişe hazırlayabilecekleri bir "Sepet Sistemi" geliştirmektir. Oturum dayanıklılığını (persistence) sağlamak ve olası bağlantı kopmalarında veri kaybını önlemek amacıyla, geçici bellek (In-Memory/Redis) çözümleri yerine **PostgreSQL veritabanı destekli (Sepet ve SepetDetay tabloları)** bir mimari tercih edilmiştir.

1-Sepet Servisi (Business Logic) ve Optimizasyon

Sepet işlemleri için `ISepetService` arayüzü ve `SepetService` sınıfı oluşturulmuştur. Bu servis içerisinde; sepete yeni ürün ekleme, var olan ürünün miktarını artırma/azaltma, opsiyonlu ürünlerin (örn: büyük boy, badem sütlü) ek fiyatlarını hesaplama ve sepeti temizleme gibi iş kuralları (business logic) kodlanmıştır. Performansı artırmak ve veri tutarsızlığını önlemek için sepet işlemleri **tek bir Transaction (işlem bütünlüğü)** üzerinden optimize edilmiştir.

```
public async Task<SepetDetay> AddItemAsync(int sepetId, int urunId, int adet, List<int>? opsiyonIds = null)
{
    var urun = await _context.Urunler.FindAsync(urunId);

    // 1. Opsiyonları bul ve ek fiyatları hesapla
    decimal opsiyonEkFiyat = 0;
    string? opsiyonJson = null;

    if (opsiyonIds != null && opsiyonIds.Count > 0)
    {
        var opsiyonlar = await _context.Opsiyonlar
            .Where(o => opsiyonIds.Contains(o.Id))
            .ToListAsync();

        opsiyonEkFiyat = opsiyonlar.Sum(o => o.EkFiyat);
        opsiyonJson = JsonSerializer.Serialize(opsiyonlar.Select(o => new { o.Id, o.Ad, o.EkFiyat }));
    }

    // 2. Birim fiyatı hesapla (Ürün + Opsiyonlar)
    var birimFiyat = urun.Fiyat + opsiyonEkFiyat;

    // 3. Mevcut ürün kontrolü veya yeni ekleme
    // ... (mevcut ürün kontrolü kodun burada devam ediyor)

    // 4. Sepet Toplam Tutarı Güncelleme (Anlık LINQ ile)
    sepet.ToplamTutar = sepet.SepetDetaylar.Sum(sd => sd.BirimFiyat * sd.Adet);

    await _context.SaveChangesAsync();
    return mevcutDetay;
}
```

2-AJAX Tabanlı Sepet Yönetimi (JSON Endpoint'ler)

Kullanıcı deneyimini kesintiye uğratmamak adına SepetController içerisindeki metotlar, tam sayfa yenilemesi (postback) yapmak yerine **AJAX (Asenkron JavaScript)** isteklerine yanıt verecek şekilde JSON tabanlı tasarlanmıştır. Bu sayede müşteri, "Sepete Ekle" butonuna bastığında sayfa yenilenmeden ürün sepete eklenmekte ve sepet tutarı anlık olarak güncellenmektedir.

```
// Güncel sepet bilgisini döndür
var guncelSepet = await _sepetService.GetSepetWithDetailsAsync(sepet.Id);
return Json(new
{
    success = true,
    message = "Ürün sepete eklendi",
    toplamTutar = guncelSepet?.ToplamTutar ?? 0,
    urunSayisi = guncelSepet?.SepetDetaylar.Sum(sd => sd.Adet) ?? 0
});
```

3 - 3.Haftada yaptığım menü tasarımı vardı basit küçük resimde ürün fotoğrafı olan isim ve sepete ekleme seçeneği bulunan yandan açılan sepet ekranı vardı ancak menü tasarımında değişikliğe gittim oyüzden ekran görüntüsü bulunmamakta ve menü tasarımı 7.hafta planı ben denemek amaçlı öne çektim ancak 7.haftada menü tasarımı tekrar revize edilecektir.

Geliştirilen bu karmaşık sepet mantığının hatasız çalıştığından emin olmak için **15 adet yeni Unit Test** (Miktar güncelleme, olmayan ürünü silme denemesi, aktif olmayan ürünü ekleme denemesi vb.) yazılmış ve projedeki toplam başarılı test sayısı 31'e çıkarılmıştır.

Hafta 4: Sipariş Motoru, State Machine ve Gerçek Zamanlı İletişim (SignalR)

Dördüncü haftada, müşterilerin oluşturduğu sepetlerin siparişe dönüşüm süreci ve siparişlerin yaşam döngüsü kodlanmıştır. Karmaşık sipariş süreçlerini yönetebilmek için 9 farklı durumu (Sepette, Hazırlanıyor, Hazır, Teslim Edildi vb.) barındıran **State Machine (Durum Makinesi)** mimarisi kurulmuştur.

1. Concurrency (Eşzamanlılık) ve İşlem Bütünlüğü

Aynı masadan birden fazla kişinin veya personelin aynı siparişe müdahale etmesi durumunda yaşanabilecek veri tutarsızlıklarını önlemek için RowVersion tabanlı **Concurrency (Eşzamanlılık)** kontrolü sağlanmıştır. Siparişin onaylanması, sepetin boşaltılması ve stok takibi gibi işlemlerin tamamı tek bir TransactionScope içerisinde atomik olarak gerçekleştirilmiştir.

2. SignalR ile Gerçek Zamanlı Haberleşme ve CI/CD Pipeline

Müşteri bir sipariş verdiğinde veya garson çağırdığında, sayfa yenilenmesine gerek kalmadan mutfak ve garson panellerine anlık bildirim düşmesi için **SignalR (MenuHub)** entegrasyonu tamamlanmıştır. Ek olarak, projenin canlı sunucuya (VDS) aktarım sürecini otomatize etmek amacıyla **GitHub Actions** kullanılarak **CI/CD Pipeline** oluşturulmuştur.

Hafta 4 Gün 2: Mobil admin fix, ürün silme FK fix, menü içeriği (38 ü...
Deploy to VDS #4: Commit 3888005 pushed by brnshn
main
Mar 10, 12:16 AM GMT+3
2m 7s

```
// Mutfaka uyarı gönder
await _menuHub.Clients.All.SendAsync("SiparisEklendi");
// Diğer panellere (Kasa, Admin) güncelleme gönder
await _menuHub.Clients.All.SendAsync("SiparisGuncellendi");
```

```
- name: 🚀 Projeyi Publish Et ve ZIP'le
  run: |
    dotnet publish src/QRMenu.Web/QRMenu.Web.csproj -c Release -r linux-x64 -o
    publish_output
    zip -r QRMenu.zip ./publish_output/*

- name: 🚀 ZIP Dosyasını VDS'e Aktar (SCP)
  uses: appleboy/scp-action@v0.1.7
  with:
    host: ${ secrets.VDS_HOST }
    source: "QRMenu.zip"
    target: "/var/www/QRMenu"
```

Hafta 5: Personel Servisleri, ACID Tahsilat ve AuditLog (Denetim İzi)

Vize öncesi son haftada, restoran operasyonlarının yönetileceği personel ekranları (Garson, Mutfak, Kasa) geliştirilmiştir. Sisteme rol bazlı (Admin, Garson, Kasa, Mutfak) yetkilendirme altyapısı entegre edilerek herkesin yalnızca kendi görev ekranına erişmesi sağlanmıştır.

1. Kasa Servisi ve Parçalı Tahsilat (ACID)

Kasa panelinde Nakit ve Kredi Kartı ayrımı yapılarak ödeme alma işlemleri kodlanmıştır. Masadaki tüm hesabı tek seferde kapatmak yerine, sadece seçilen ürünlerin hesabını ödemeye imkan tanıyan **Parçalı Ödeme** sistemi OdemeService içerisine dahil edilmiştir. Bu işlemler finansal veriler içerdiği için katı ACID prensiplerine sadık kalınarak gerçekleştirilmiştir.

✓ 1 xAmericano
Orta
Durum: TeslimEdildi

65,00 TL

Seçilen Tutar

65.00 TL

Nakit

Kredi Kartı

2. AuditLog Interceptor ve Performans Optimizasyonu

Sistemdeki kritik değişiklikleri (örneğin bir fiyatın değişmesi veya onaylı bir siparişin iptali) takip edebilmek adına, Entity Framework Core **SaveChangesInterceptor** kullanılarak otomatik bir AuditLog (Denetim İzi) mekanizması kurulmuştur. Bu sayede her değişiklik, yapan kullanıcı ve tarih bilgisiyle veritabanına loglanmaktadır. Ayrıca, veritabanı performansını artırmak için ürün görselleri veritabanından alınarak dosya sistemine (wwwroot/uploads) taşınmış, sistem yükü minimize edilmiştir.

```
public override ValueTask<InterceptionResult<int>> SavingChangesAsync(
    DbContextEventData eventData,
    InterceptionResult<int> result,
    CancellationToken cancellationToken = default)
{
    var context = eventData.Context;
    if (context == null) return base.SavingChangesAsync(eventData, result,
        cancellationToken);

    // Değişen tüm kayıtları (Insert, Update, Delete) bul
    var entries = context.ChangeTracker.Entries()
        .Where(e => e.State == EntityState.Added || e.State == EntityState.Modified ||
        e.State == EntityState.Deleted)
        .ToList();

    foreach (var entry in entries)
    {
        var auditlog = new AuditLog
        {
            TabloAdi = entry.Entity.GetType().Name,
            Islem = entry.State.ToString().ToUpperInvariant(),
            IslemTarihi = DateTime.UtcNow,
            KullaniciId = _currentUserProvider.GetUserId(),
            // Eski ve Yeni değerleri JSON formatında otomatik yakalar
            EskiDeger = entry.State != EntityState.Added ?
            JsonSerializer.Serialize(entry.OriginalValues.ToObject()) : null,
            YeniDeger = entry.State != EntityState.Deleted ?
            JsonSerializer.Serialize(entry.CurrentValues.ToObject()) : null
        };

        context.Set<AuditLog>().Add(auditLog);
    }
}
```



Garsonun İndex Ekranıdır Dolu Masa + Sipariş Uyarısı Ve Ürün Sayısı Listelenir

Yeni Sipariş (Masa 2) X

Sıcak İçecekler

Filtre Kahve
45,00 TL

+

Latte
65,00 TL

+

Americano
55,00 TL

+

Çay
20,00 TL

+

Sahlep
50,00 TL

+

Seçili Ürünler

Filtre Kahve (x1)

-

+

Ice Latte (x1)

-

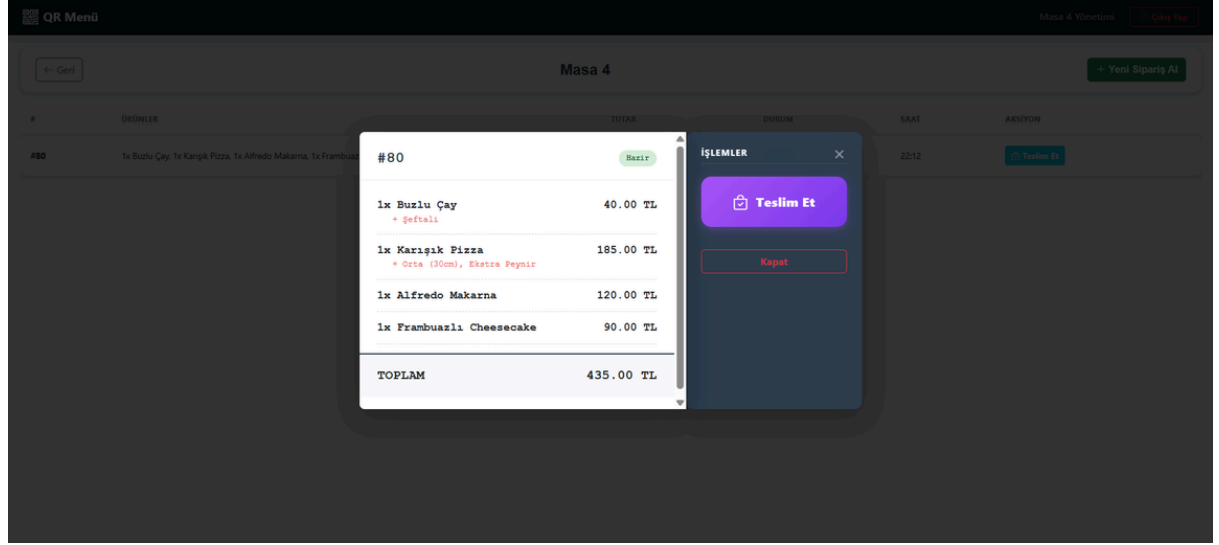
+

Siparişi Onayla (115.00 TL)

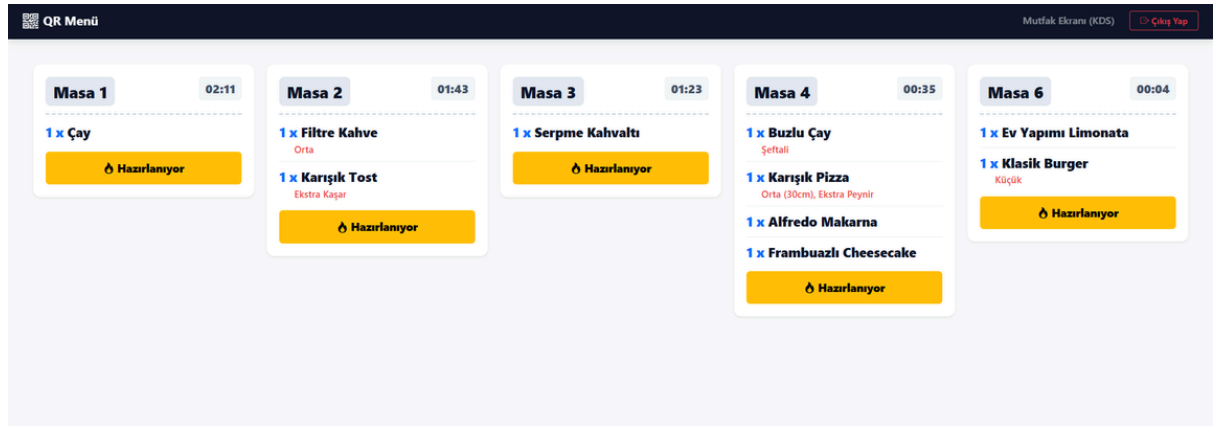
[22:22:19 INF] Garson tarafından manuel sipariş oluşturuldu. SiparisId=82, MasaId=2, Tutar=115.00

Garson Qr çalışmayan veya telefon sorunu olan müşteriler için kendi panelinden

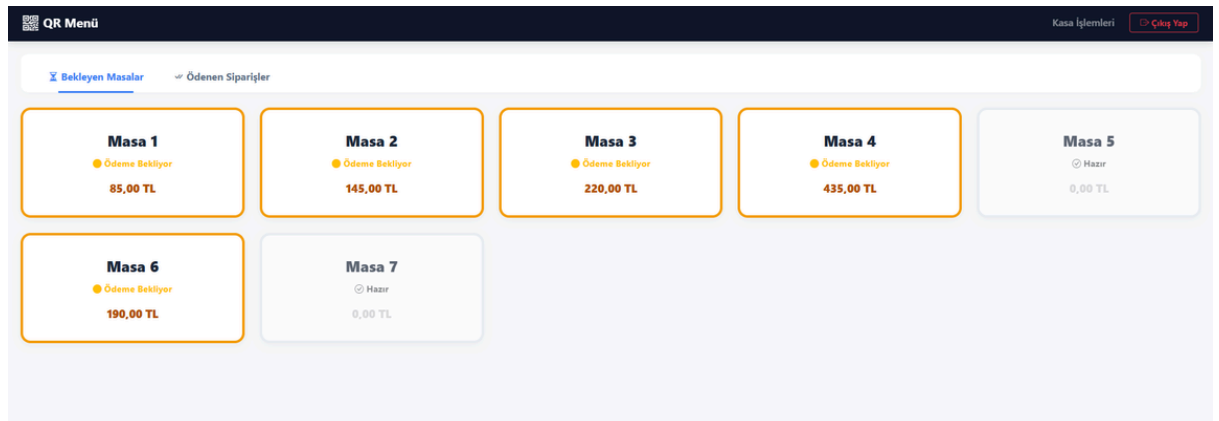
masaya girip sipariş oluşturabiliyor.



Masaya Girdikten Sonra Siparişe Tıklanıp Fiş Görüntülenir ve Teslim Etme Seçeneği Bulunuyor.



Mutfak Ekranının Siparişleri Listelenmiş Örnek Halidir Bu ekranın tasarımını aynı garsondaki fiş ile değiştirmeyi düşünüyorum.



Kasanın Gördüğü Ekrandır Masalar içeriğindeki ödemeler aktif olmayan masalarında içeriğine girilemiyor çünkü öyle bir masa yok varsayıyor. Ödeme varsa ödeme bekleniyor uyarısı verir.

QR Menü

Masa 4 Tahsilat

Çıkış Yap

Geri

Masa 4 Tahsilatı

Tümünü Seç

Adisyon #80 - 19:12

☒ 1 x Buzlu Çay

Şeftali

Durum: Teslim Edildi

40,00 TL

☐ 1 x Karşık Pizza

Orta (Döner) Ekstra Peynir

Durum: Teslim Edildi

185,00 TL

☒ 1 x Alfredo Makarna

Durum: Teslim Edildi

120,00 TL

☐ 1 x Frambuazlı Cheesecake

Durum: Teslim Edildi

90,00 TL

Seçilen Tutar

160.00 TL

Nakit

Kredi Kartı

Ödemede bu şekilde tekli çoklu seçim yapılabiliyor ve ödeme seçeneğinde nakit veya kart ödendi diyerek sipariş kapatılıyor.